Konwolucyjne sieci neuronowe

CNN – Convolutional Neural Networks

Dariusz Put

Na podstawie:
https://brohrer.github.io/how_convolutional_neural_networks_work.html

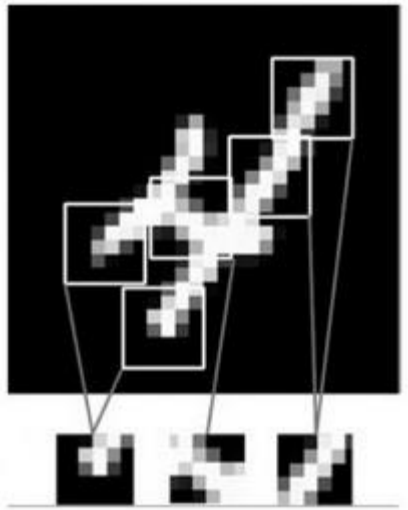
Konwolucyjne (splotowe) sieci neuronowe (ang. *Convolutional Neural Networks* – CNN, **convnet**).

Od 2012 roku stały się jednym z najpopularniejszych algorytmów do przetwarzania obrazu, sekwencji wideo i tekstu.

Sprawdzają się podczas modelowania wszystkich zadań percepcyjnych.

Są wzorowane na działaniu mózgu w procesie rozpoznawania obrazów.

Podstawową różnicą między warstwą gęstych połączeń (Dense) a siecią konwolucyjną jest to, że warstwy gęste uczą się cech parametrów globalnych w swoich przestrzeniach wejściowych (w przypadku MNIST są to wzorce związane ze wszystkimi pikselami), a warstwy konwolucyjne uczą się **lokalnych wzorców**.



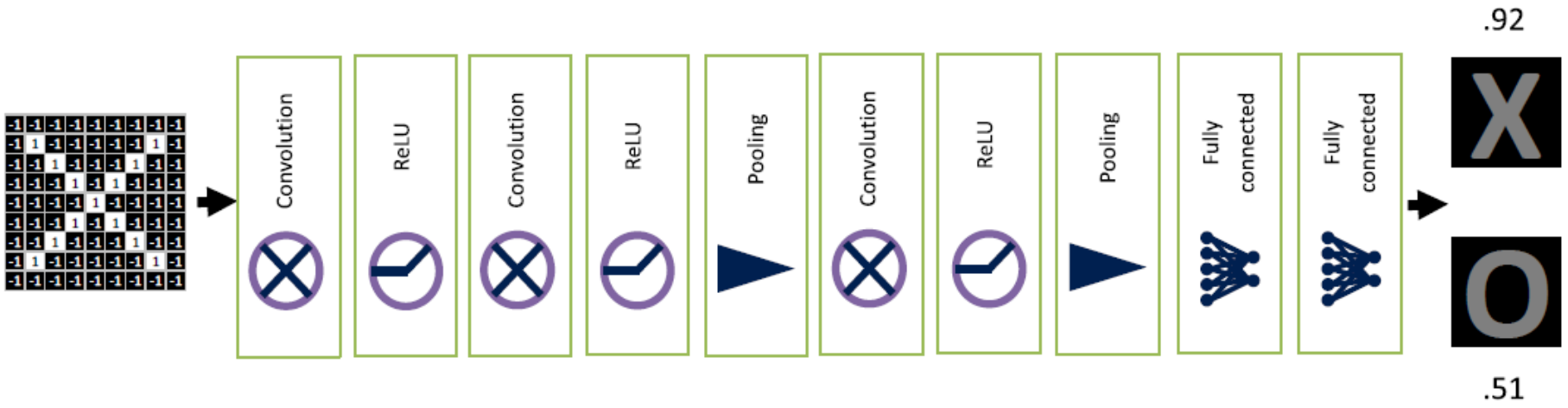
Dzięki temu sieci konwolucyjne mają dwie istotne własności:

- wzorce rozpoznawane przez sieć są **niezależne od przesunięcia**. Dzięki temu sieci konwolucyjne charakteryzują się dużą wydajnością podczas przetwarzania obrazów.
- sieci konwolucyjne mogą uczyć się **przestrzennej hierarchii wzorców**: pierwsza warstwa uczy się małych lokalnych wzorców (krawędzie), druga większych struktur itd.

W odróżnieniu od tradycyjnych sieci neuronowych, w CNN występuje większa różnorodność typów warstw:

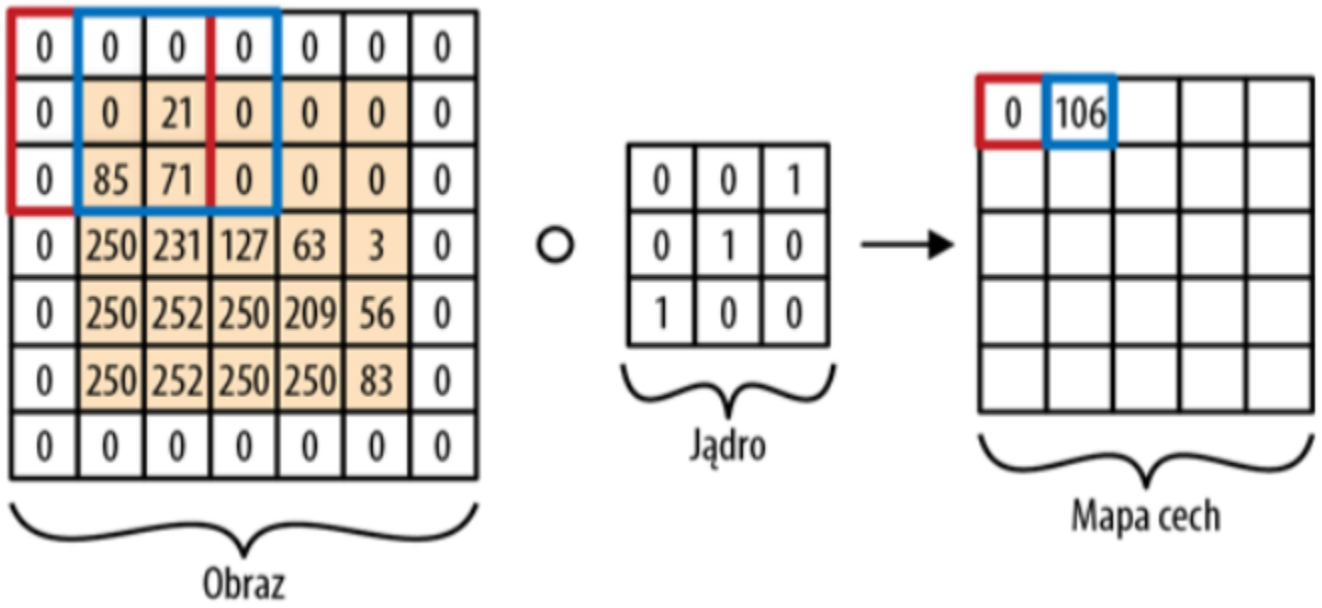
- splotowa (*convolution*)
- ReLU
- łącząca (*pooling*)
- porzucania (*dropout*)
- spłaszczająca (*flatten*)
- gęsta (*fully connected*, *dense*)

Wykorzystując je, tworzymy głęboką sieć neuronową CNN, która zbiór pikseli przekształca w zbiór „głosów” wskazujących prawdopodobieństwo tego, że dany obraz należy do danej kategorii.



Operacja **splotu** (konwolucji) polega na przesuwaniu jednego elementu, tzw. jądra (ang. *kernel*) po innym elemencie (fragmencie analizowanego obrazu).

Jądro splotowe to macierz wag. Jest to zwykle mała, dwuwymiarowa tablica. Wagi jądra są mnożone element po elemencie przez odpowiadające im liczby w lokalnym polu recepcyjnym, a wyniki mnożenia są sumowane.
W ten sposób następuje transformacja obrazu i powstaje mapa cech o zredukowanym rozmiarze.





0	0	0	0	0	30	0
0	0	0	0	30	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	0	0	0	0

Wzorzec

Analizowany fragment obrazu



0	0	0	0	0	0	30
0	0	0	0	50	50	50
0	0	0	20	50	0	0
0	0	0	50	50	0	0
0	0	0	50	50	0	0
0	0	0	50	50	0	0
0	0	0	50	50	0	0

*

0	0	0	0	0	30	0
0	0	0	0	30	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	0	0	0	0

(50*30)+(50*30)+
(50*30)+(20*30)+(50*30) =6600
Fragment bardzo pasuje do wzorca – wynikiem mnożenia i dodawania jest duża liczba.



0	0	0	0	0	30	0
0	0	0	0	30	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	0	0	0	0

Wzorzec

Analizowany fragment obrazu



0	0	0	0	0	0	0
0	40	0	0	0	0	0
40	0	40	0	0	0	0
40	20	0	0	0	0	0
0	50	0	0	0	0	0
0	0	50	0	0	0	0
30	30	0	50	0	0	0

*

0	0	0	0	0	30	0
0	0	0	0	30	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	0	0	0	0

Fragment nie pasuje do wzorca – wynikiem mnożenia i dodawania jest mała liczba (tu równa 0).

Konwolucje są definiowane przez dwa główne parametry:

1. Rozmiar jądra (zwanego także łatą lub oknem): najczęściej 3x3, rzadziej 5x5 lub 7x7
2. Głębia wyjściowej mapy cech – liczba filtrów obliczana przez konwolucję

W pakiecie Keras warstwa konwolucyjna (Conv2D) wymaga podania tych parametrów w następujący sposób:
Conv2D(głębia_wyjścia, (wysokość_okna, szerokość_okna))

np.:
Conv2D(32, (3,3), …
gdzie:
32 – liczba filtrów obliczana przez konwolucję
(3,3) – rozmiar okna

Do wyznaczenia wag jąder przez sieć splotową wykorzystywana jest metoda propagacji wstecznej, podobnie, jak w tradycyjnej sieci neuronowej.
Do inicjalizowania wartości wag jąder wykorzystywane są te same metody, co do inicjalizowania wag neuronów w tradycyjnych sieciach neuronowych (np. w pakiecie keras jest to inicjalizacja Glorota).

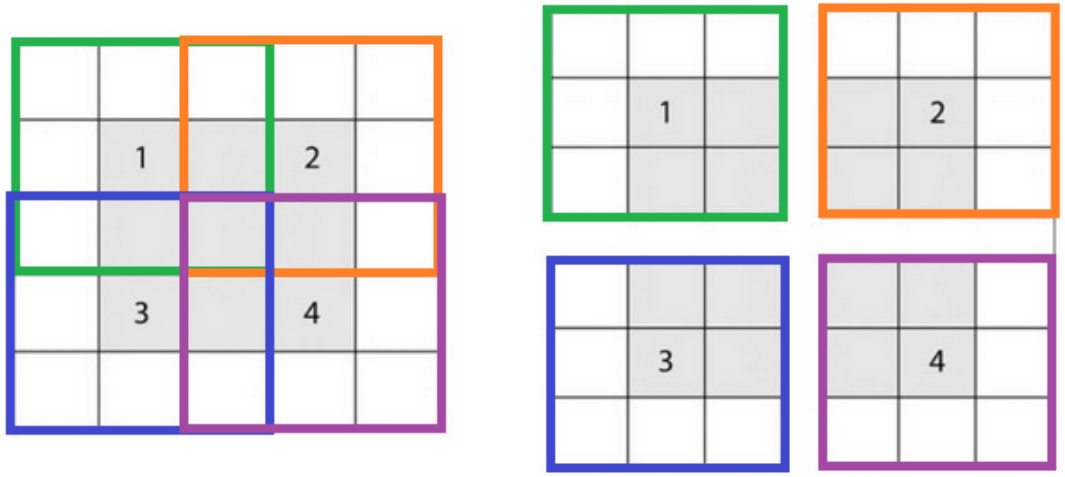
Konwolucja polega na przesuwaniu okna po mapie cech. Szerokość i wysokość obiektu wyjściowego może się różnić od szerokości i wysokości obiektu wejściowego.

Może to wynikać z dwóch czynników:

- efektów granicznych, którym można przeciwdziałać dopełniając mapę cech wyjściowych,
- użycia **rozmiaru kroku** (ang. *stride size*) oznaczającego, co ile mają się przesuwać okna (zwykle 1). Jeśli wartość ta jest większa od 1, otrzymujemy efekt tzw. konwolucji kroczących.

Łaty konwolucji 3x3 przy kroku 2x2

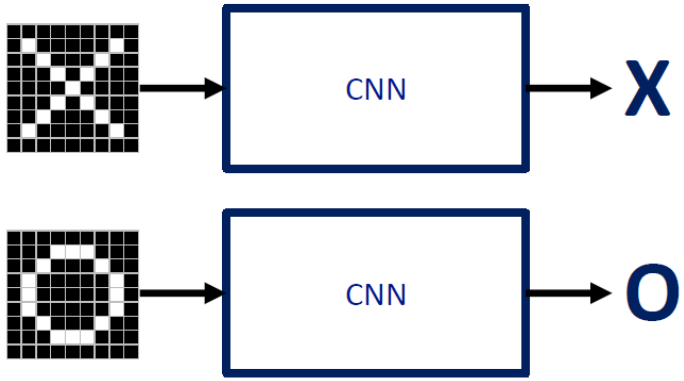
wejście:
5x5



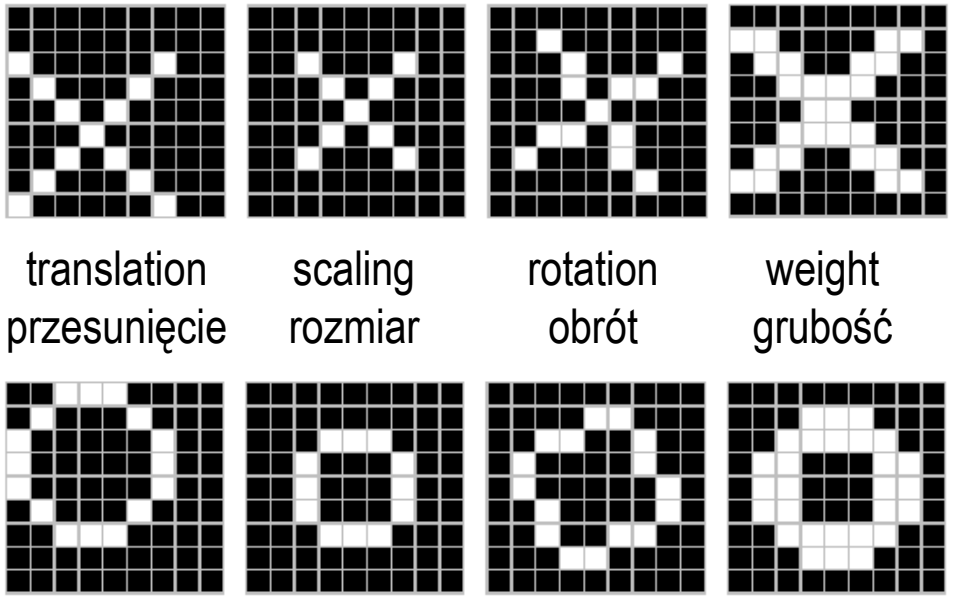
wyjście:
2x2

Wzrost kroku prowadzi do zmniejszania rozmiarów mapy cech wyjściowych.

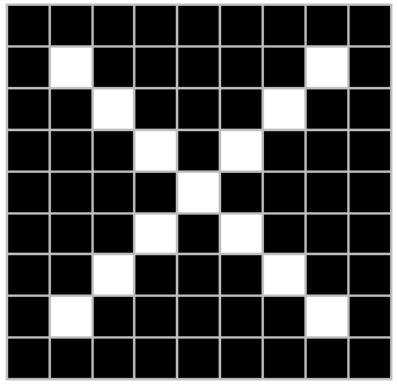
Przykład klasyfikacji binarnej:
zadaniem jest rozpoznanie, czy na rysunku jest kółko, czy krzyżyk.



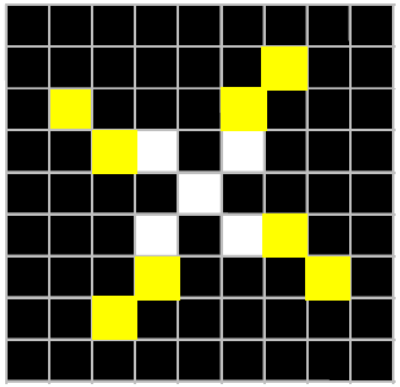
Rysunki mają różną formę:



Komputer porównuje piksele. Na rysunku z prawej liczba pikseli niezgodnych z wzorcem (z lewej) jest większa, niż zgodnych. Człowiek potrafi rozpoznać, że na rysunku z prawej jest krzyżyk. Komputer niekoniecznie. CNN pozwalają mu „zmądrzeć”.



wzorzec

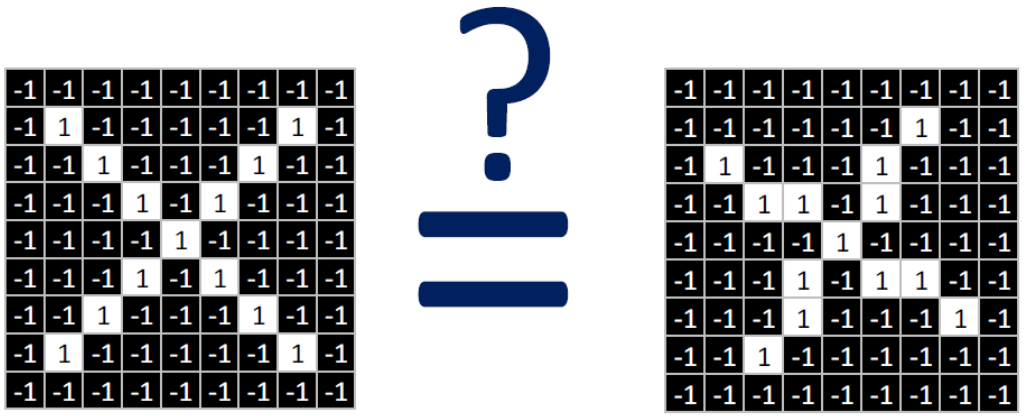


rozpoznawany obraz

Liczba pikseli zgodnych: 5 (kolor biały)
Liczba pikseli niezgodnych: 8 (kolor żółty)

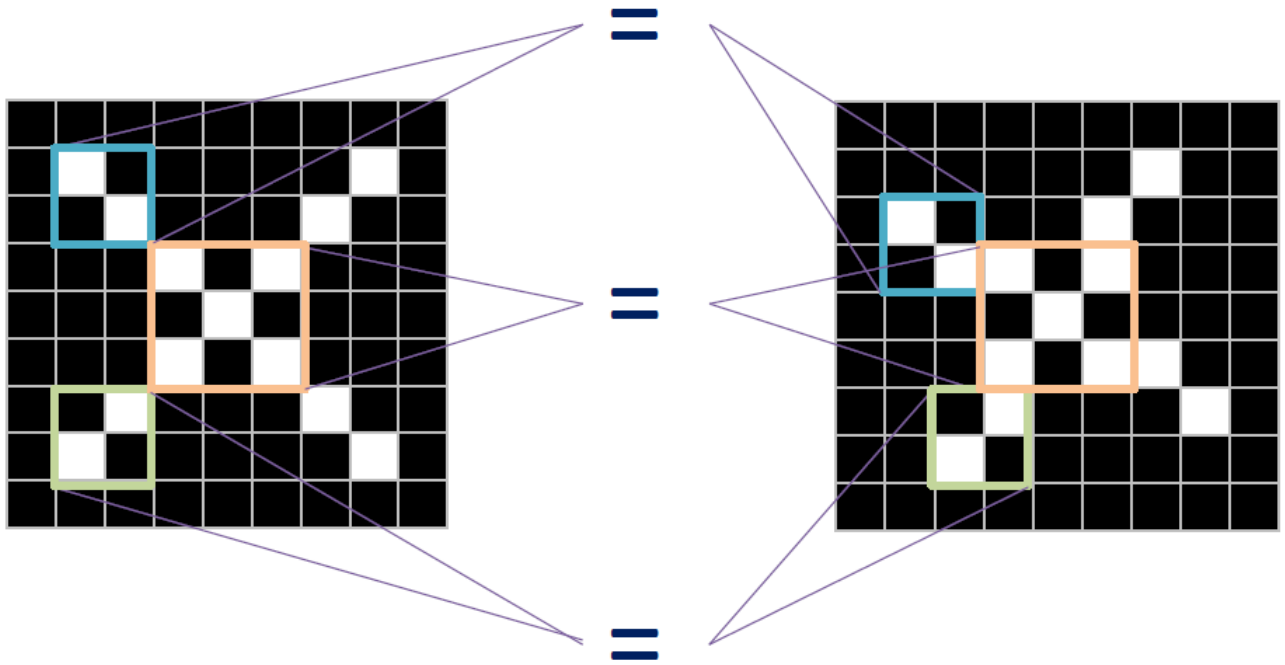
Komputer rozróżnia tylko 0 i 1.

Załóżmy, że piksel biały jest reprezentowany przez 1 a czarny przez -1:

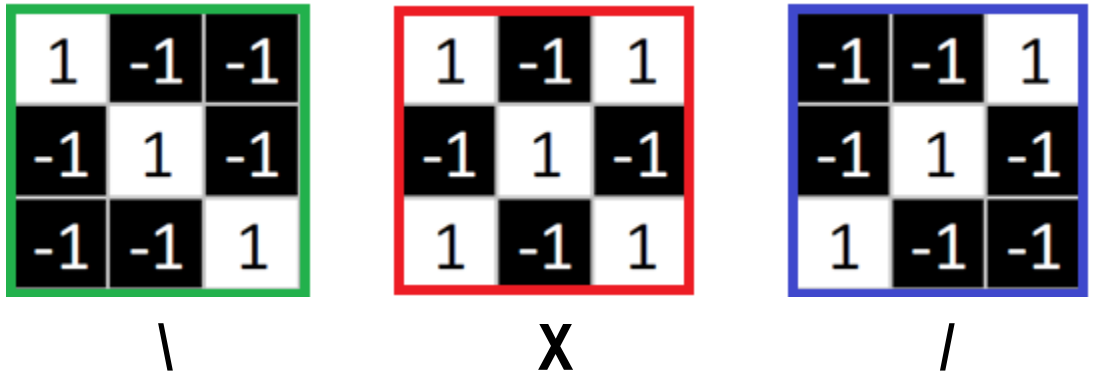


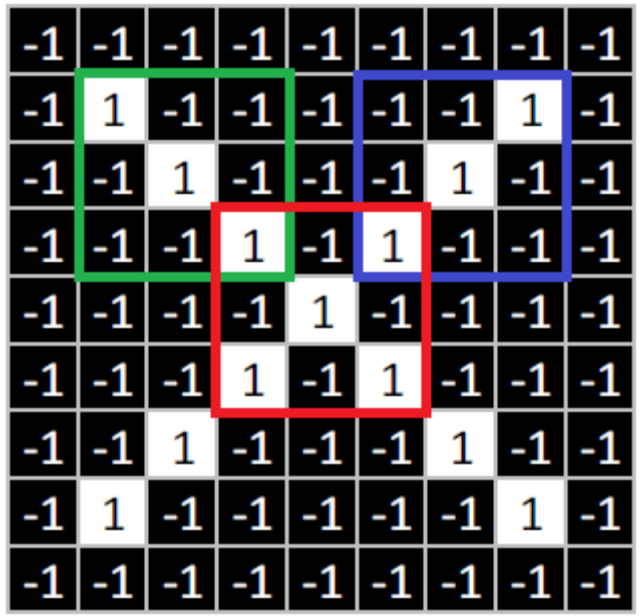
wzorzec rozpoznawany obraz

Konwolucyjne sieci neuronowe (CNN) dopasowują fragmenty obrazów.



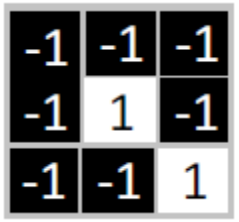
Zajmijmy się problemem rozpoznania, czy na rysunku jest **krzyżyk**. Będziemy poszukiwali trzech wzorców – będą to nasze **filtry**:



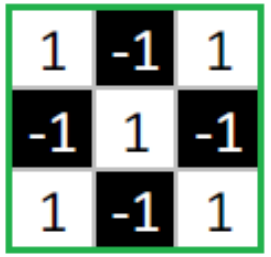


Algorytm postępowania:

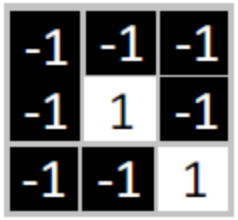
1. Sprowadź filtr i analizowany fragment do postaci liniowej.
2. Pomnóż korespondujące piksele.
3. Zsumuj iloczyny.
4. Podziel otrzymany wynik przez liczbę pikseli, otrzymasz prawdopodobieństwo tego, że analizowany fragment pasuje do wzorca.



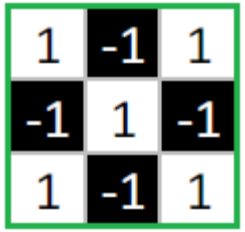
analizowany fragment



wzorzec (filtr)



analizowany fragment



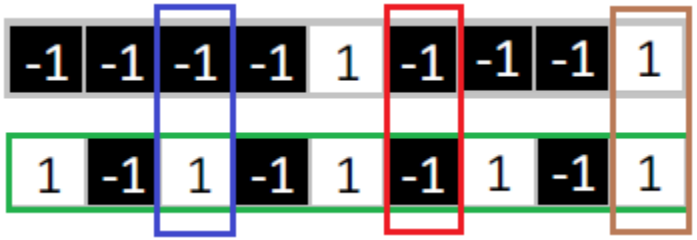
wzorzec



1. Postać liniowa



2-3. Mnożenie i dodawanie



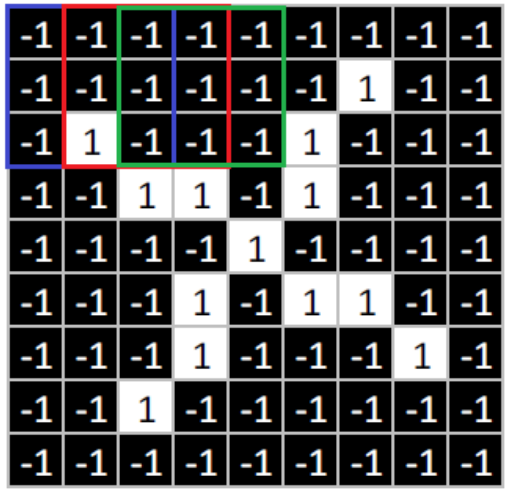
(-1)*1 + (-1)*(-1) + (-1)*1 + (-1)*(-1) + 1*1 + (-1)*(-1) + (-1)*1 + (-1)*(-1) + 1*1 = -1 + 1 - 1 + 1 + 1 + 1 - 1 + 1 + 1 = 3

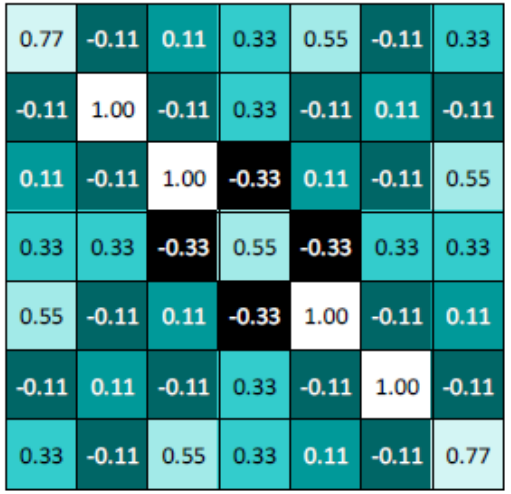
4. Obliczanie prawdopodobieństwa

3 / 9 = 0.3333

Prawdopodobieństwo, że analizowany fragment pasuje do wzorca wynosi 0.3333

Operację tę wykonujemy dla każdego fragmentu analizowanego obrazu i otrzymujemy **mapę dopasowania**.





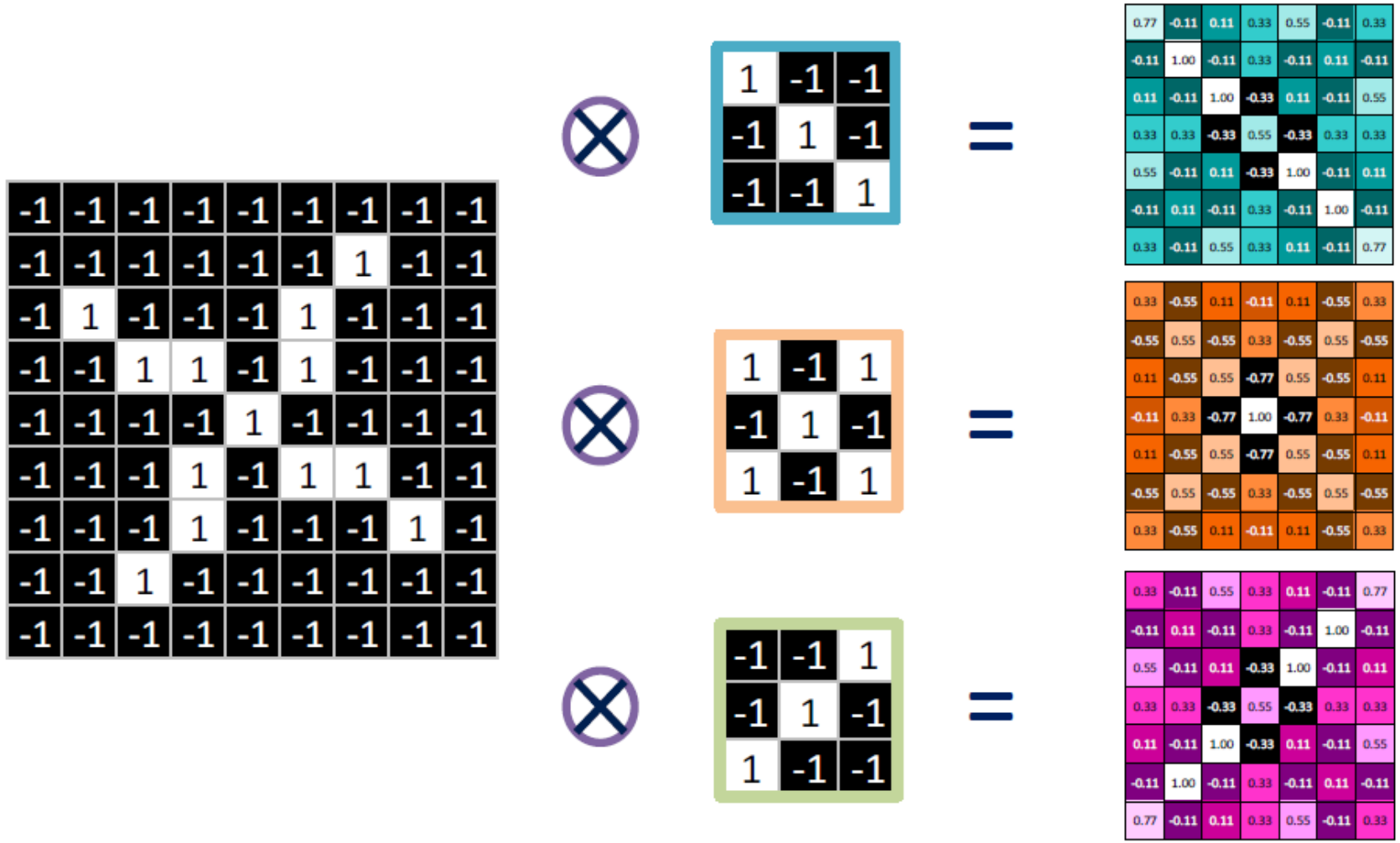
Mapa dopasowania:

1 – maksymalne dopasowanie

-1 – maksymalne dopasowanie w negatywie

0 – brak dopasowania

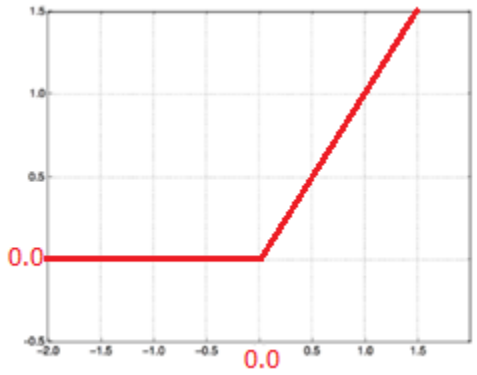
Jeśli tą samą operację wykonamy dla wszystkich wzorców, otrzymamy trzy mapy dopasowania. Jeden obraz staje się na wyjściu zbiorem przefiltrowanych obrazów.



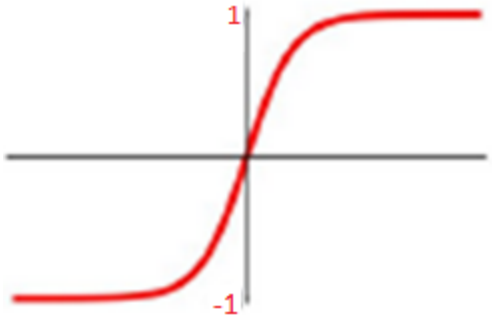
W przypadku dużej liczby wzorców (filtrów) na wyjściu będzie generowana duża liczba obrazów, co może powodować problemy z wydajnością.

Normalizacja – przekształcanie danych zgodnie z wybraną funkcją normalizacyjną.

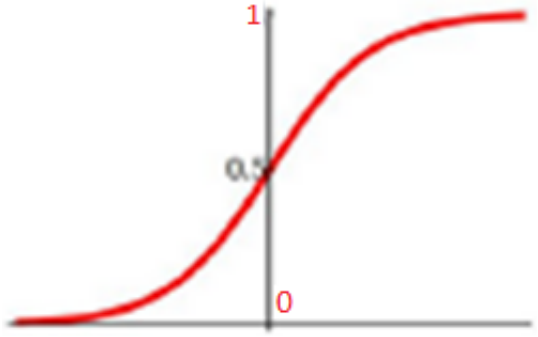
ReLU (Rectified Linear Unit)
(zamiana wartości ujemnych na 0)



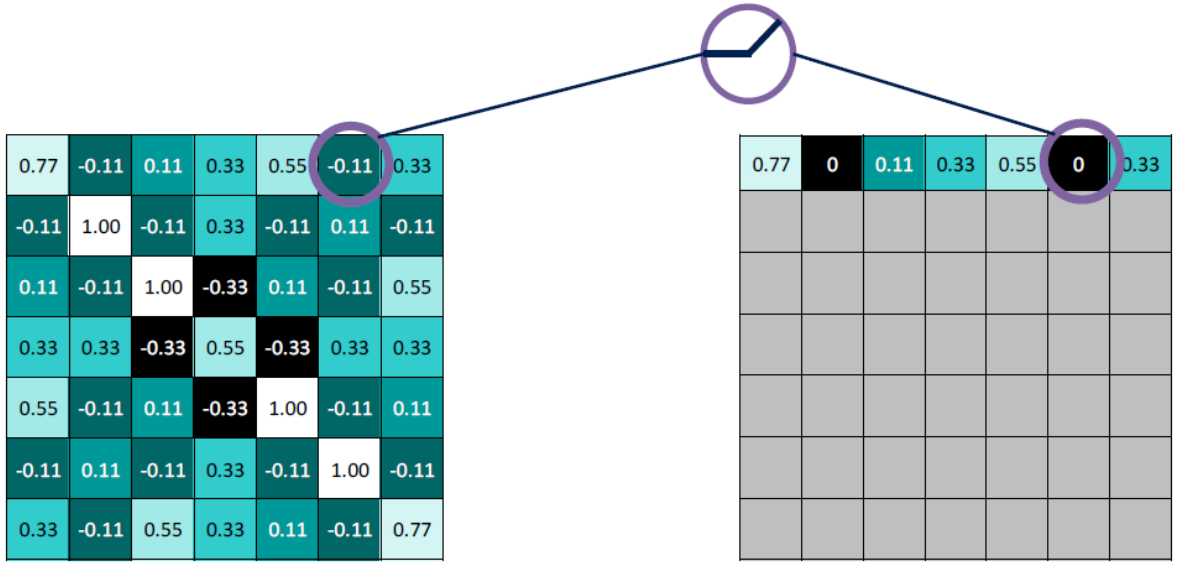
Tangens hiperboliczny (**Tanh**)
– standaryzacja do zakresu <-1; 1>



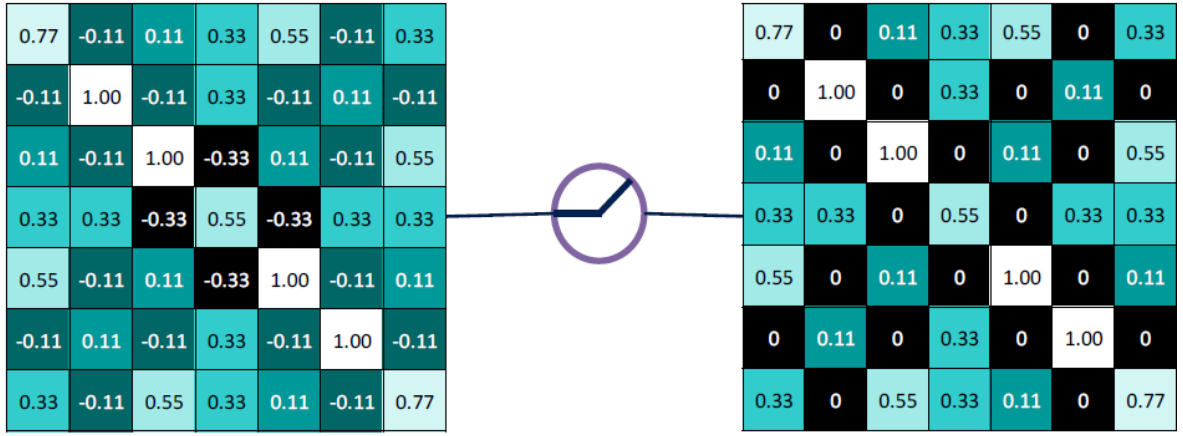
Funkcja logistyczna
(sigmoidalna – **Sigmoid**)
standaryzacja do zakresu <0; 1>



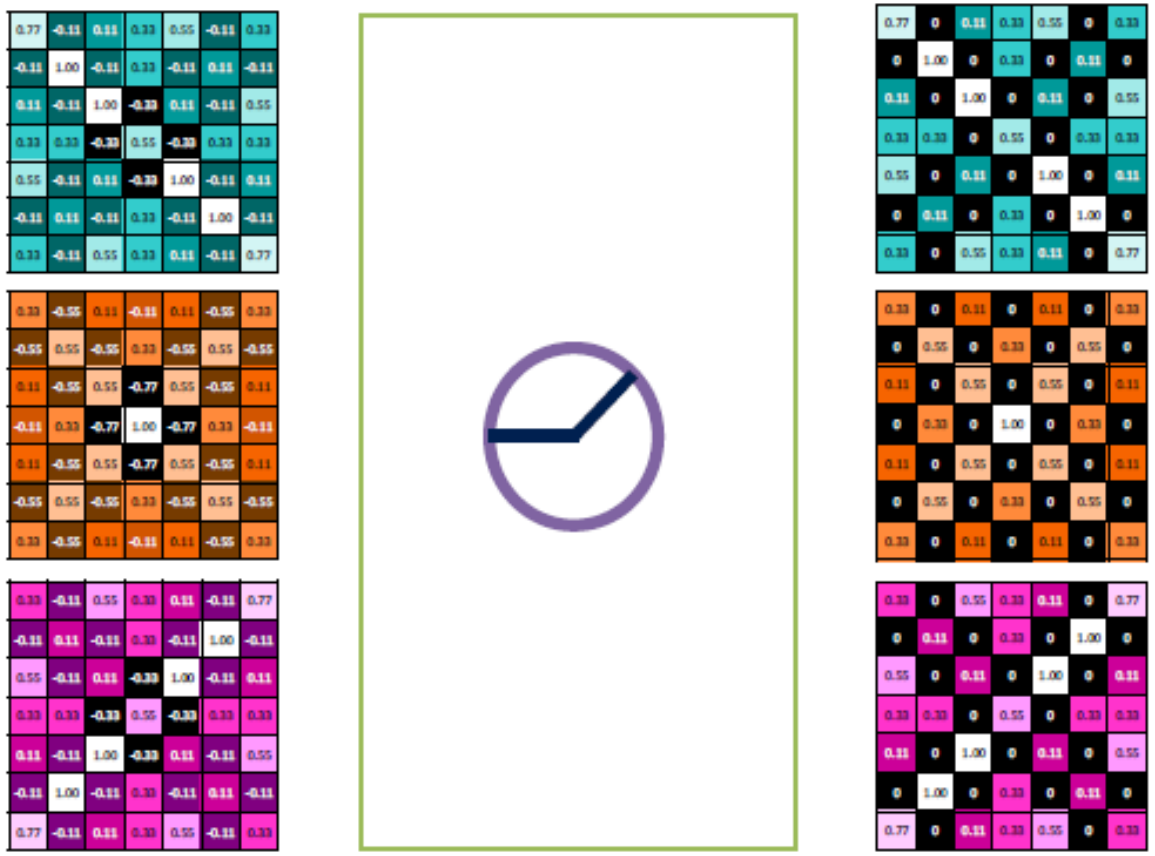
Przykład normalizacji z wykorzystaniem funkcji ReLU:



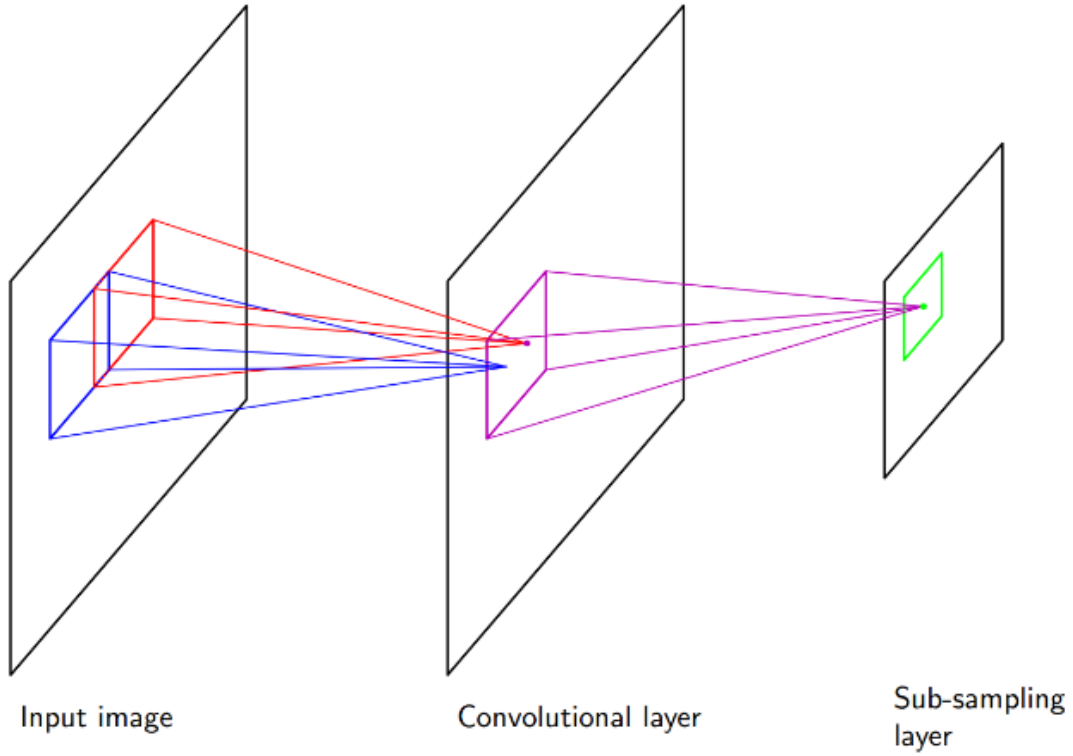
Efekt końcowy – wszystkie ujemne wartości zostały wyzerowane.



Po zastosowaniu normalizacji ReLU otrzymujemy mapy dopasowania bez wartości ujemnych.



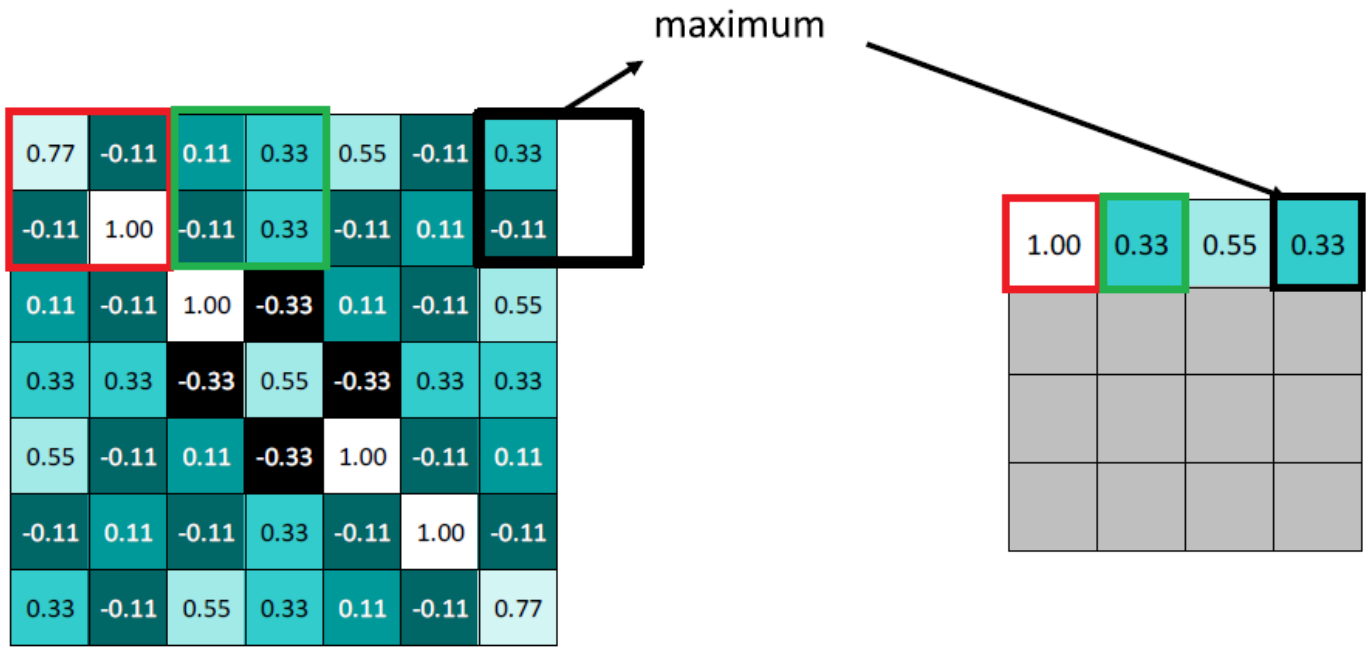
W sieciach neuronowych po warstwie konwolucyjnej występuje często warstwa **pooling** (zwana też warstwą **sub-sampling**).



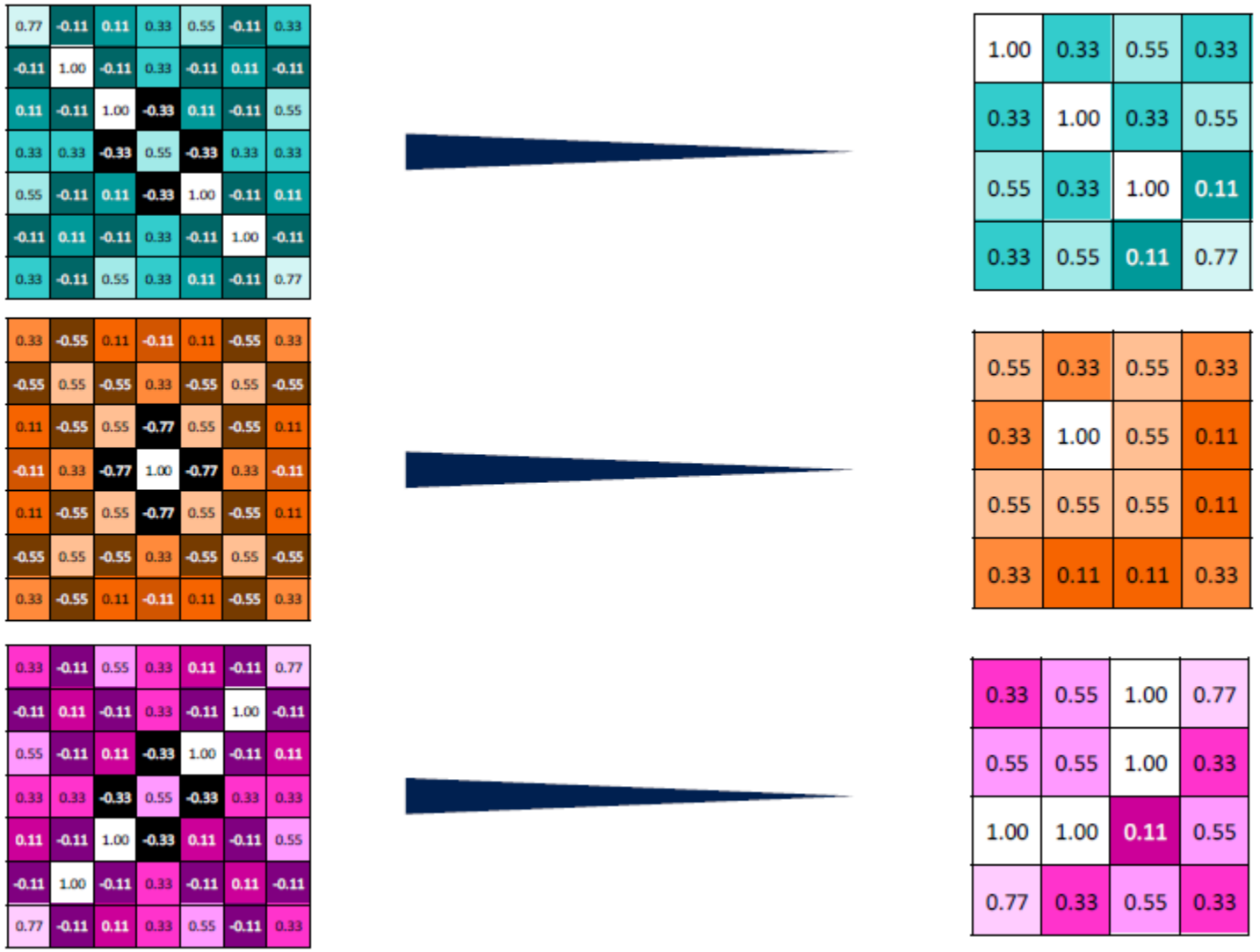
Operacja **poolingu** ma na celu zmniejszenie rozmiarów map dopasowania poprzez uogólnienie informacji wydobytych przez warstwę konwolucyjną. Obchodzi się w ten sposób problem zbyt literalnego podchodzenia przez komputer do porównywania obrazów. Traci się część informacji, ale okazuje się, że nie ma to znaczenia dla efektów końcowych, natomiast zwiększa się szybkość uczenia.

Pooling przebiega według następującego algorytmu:

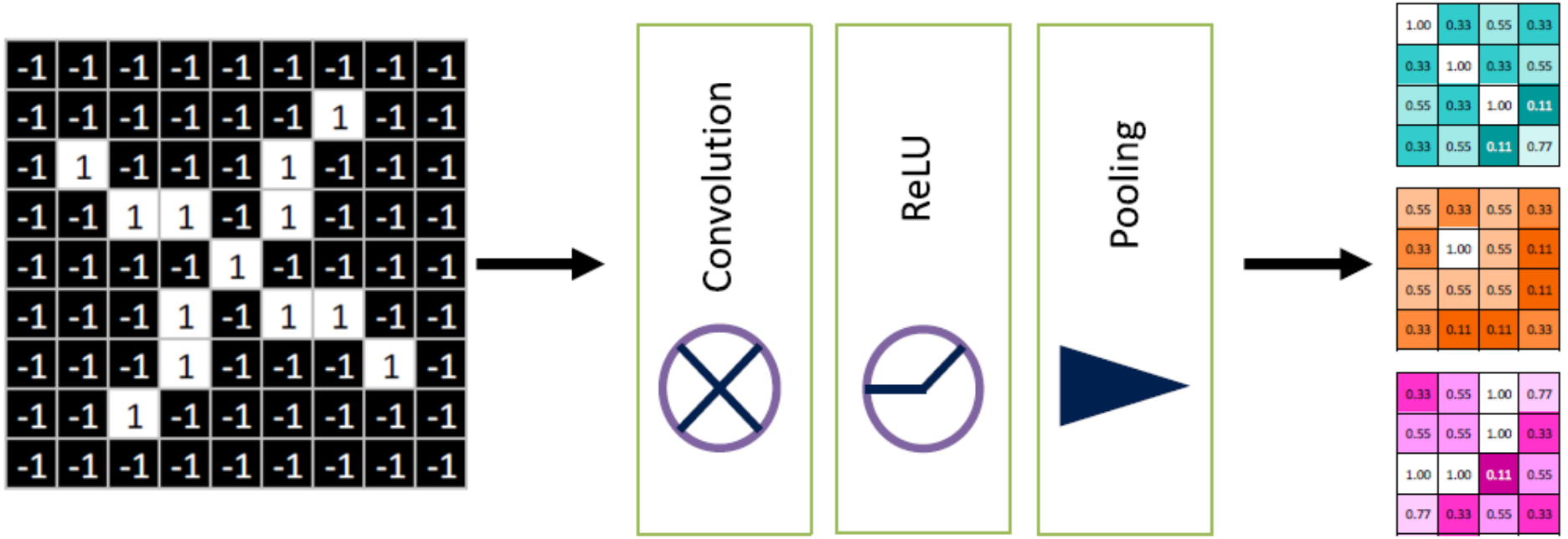
1. Wybierz rozmiar okna (zwykle 2 lub 3).
2. Wybierz krok (zwykle 2).
3. Przejdź oknem przez całą mapę przesuwając je o tyle pól, ile wynosi krok.
4. Z każdego okna wybierz wartość maksymalną (dlatego ta warstwa nazywana jest **max pooling layer**).



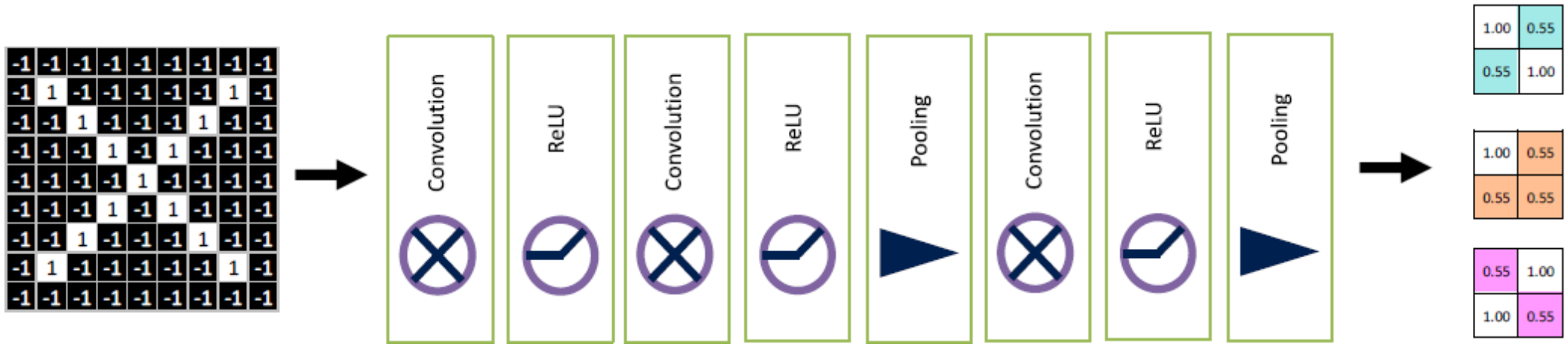
Efekt operacji poolingu dla trzech map dopasowania (rozmiar okna: 2, krok: 2).



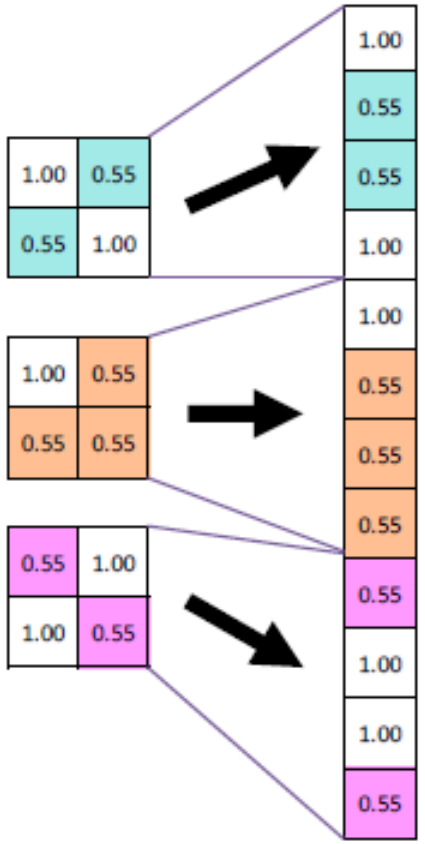
Połączenie wszystkich trzech technik. Wyjście z jednej operacji staje się wejściem dla drugiej.



Zaletą sieci wielowarstwowych jest to, że mogą się uczyć szczegółów na różnym poziomie abstrakcji. Na przykład, jeśli będziemy trenować sieć CNN do klasyfikacji obrazów, pierwsza warstwa zostanie wytrenowania do rozpoznania podstawowych elementów, takich jak krawędzie, następna będzie rozpoznawać kolekcje krawędzi, kolejna kształty, takie jak koła, nogi, ogony, twarze a następna całe obiekty (samochody, okręty, psy, ptaki, itp.).



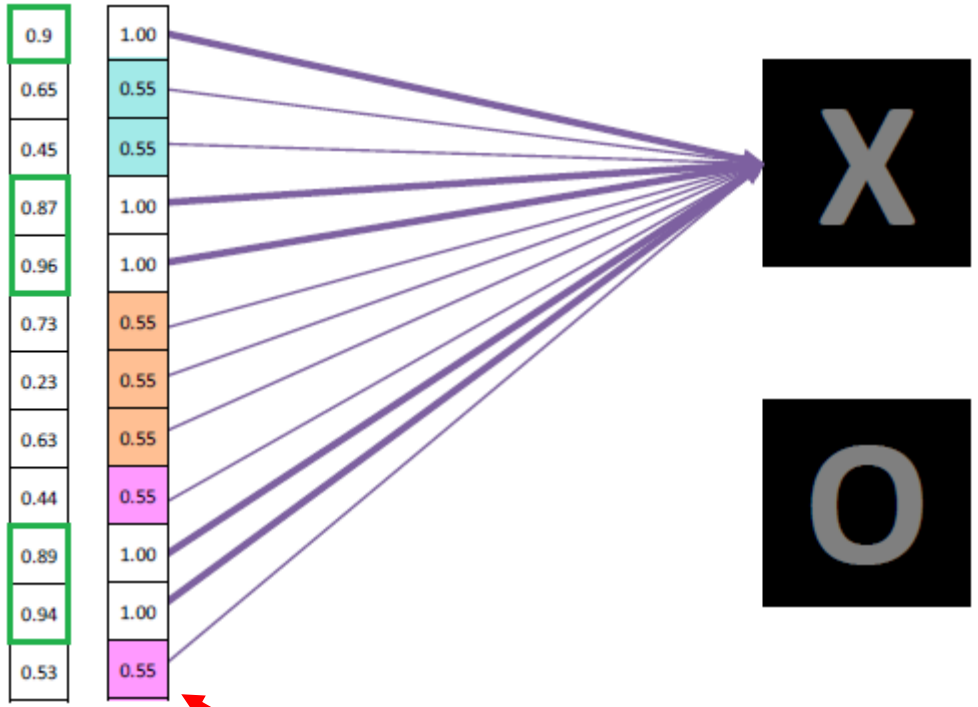
Następnie obrazy wyjściowe są przekształcane do postaci wektora.



Spłaszczone wyjście dla obrazu, na którym jest **krzyżyk**.
Pięć neuronów ma wysokie wartości.

Warstwa w pełni połączona (**fully connected layer**) to warstwa, która posiada wszystkie możliwe połączenia między jej neuronami a wyjściami neuronów warstwy poprzedniej.

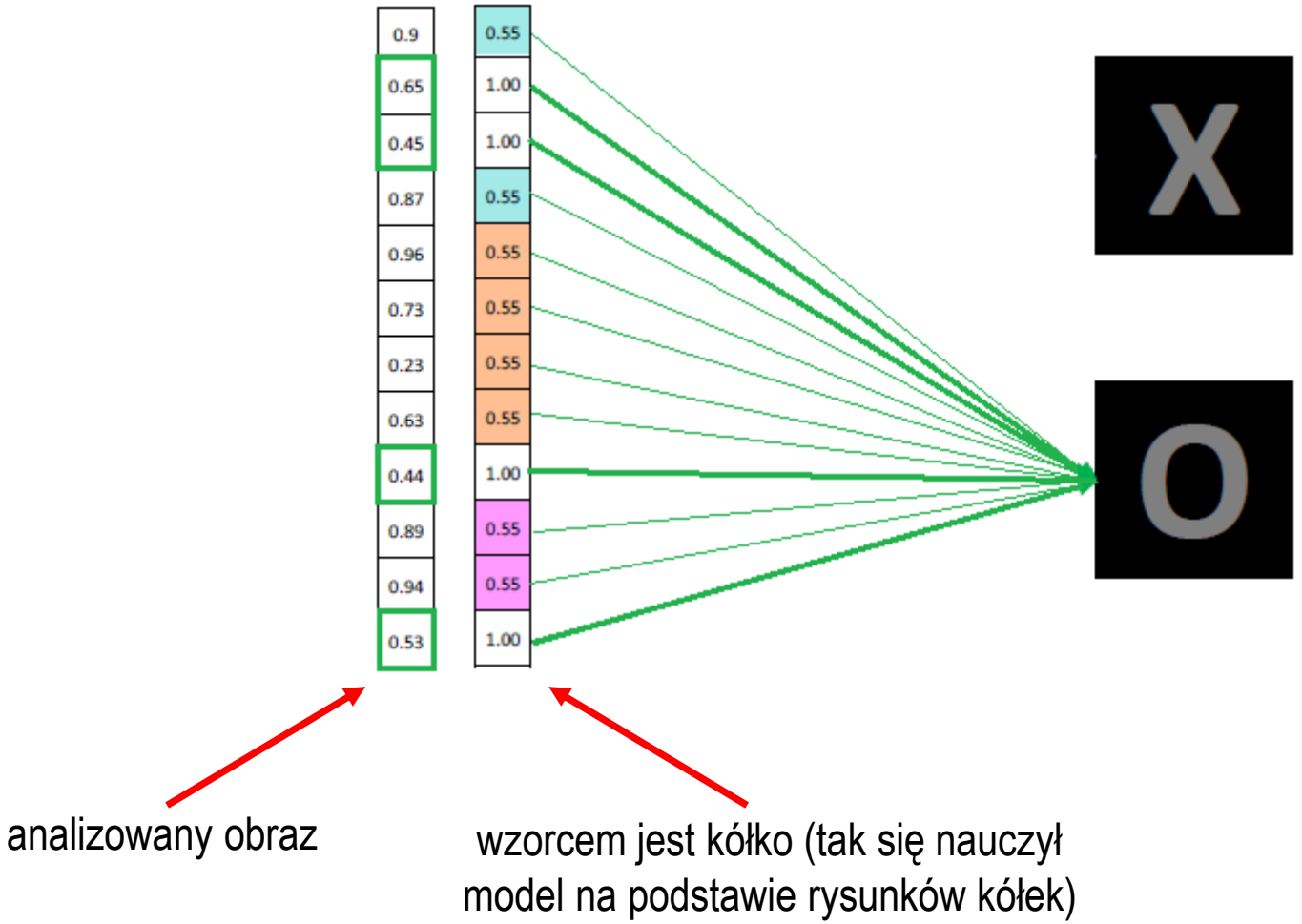
Warstwa w pełni połączona pobiera przefiltrowane dane, i każdemu elementowi przypisuje wagę oznaczającą, z jakim prawdopodobieństwem ten element wskazuje poprawny wynik.
Np. prawdopodobieństwo, że obraz jest krzyżykiem wynosi (0.9+0.87+0.96+0.89+0.94)/5=**0.92**:



analizowany obraz (nie wiemy co na nim jest, sprawdzamy, czy jest to krzyżyk)

z poprzedniego slajdu (dla krzyżyka) – tak się nauczył model na podstawie rysunków krzyżyków

Prawdopodobieństwo, że obraz jest kółkiem wynosi (0.65+0.45+0.44+0.53)/4=**0.51**.
Oczywiście kategorii obrazów może być więcej.

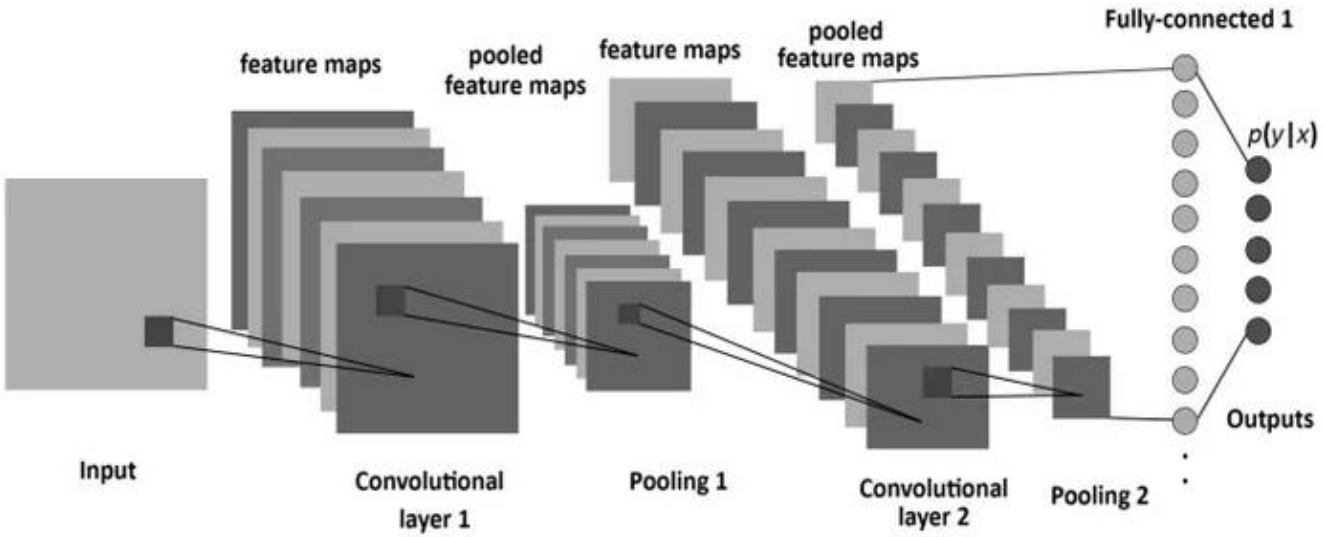


Głębokie sieci CNN są trudne do wytrenowania, gdyż:

- wymagania odnośnie liczby danych rosną wraz ze wzrostem głębokości sieci (liczby warstw),
- ze wzrostem liczby parametrów, regularyzacja sieci staje się coraz bardziej istotna, pozwalając na przeniesienie akcentów nauki z pamiętania (*memorization*) cech do uogólniania (*generalization*). Regularyzacja polega na dodaniu dodatkowej informacji pozwalającej zabezpieczyć sieć przed przeuczeniem.

Przykładowa sieć CNN składająca się z czterech bloków:

(Convolution layer → Pooling layer) *2 → Flattening layer → Dense/Output layer



Warstwa konwolucyjna składa się z trzech operacji:

- konwolucji (*convolution*),
- aktywacji (*activation*),
- normalizacji wsadu (*batch normalization*).

Czasami, w celu regularyzacji, po warstwie pooling występuje warstwa **Dropout**.
Po warstwie **Flattening** (przed finalną warstwą Dense) może zostać użytych wiele warstw **Dense**.

W sieciach konwolucyjnych czasem **zwiększa się wymiary kanałów warstw sieci** (zwiększa się rangi tensorów), co daje kilka korzyści:

1. **Ekstrakcja cech**: Kanały w warstwach konwolucyjnych służą do ekstrakcji różnych cech obrazu. Każdy kanał może reprezentować różne aspekty obrazu, takie jak krawędzie, tekstury, kolory itp. Dzięki większej licznie kanałów sieć może wydobywać większą ilość informacji z obrazów.
2. **Zwiększona reprezentatywność**: W miarę zwiększania liczby kanałów sieć ma możliwość tworzenia bardziej skomplikowanych cech, co z kolei może prowadzić do lepszej reprezentacji danych i poprawy wydajności modelu.
3. **Regularyzacja**: Wielkość kanałów może działać jako rodzaj regularyzacji. Większa liczba kanałów może pomóc w redukcji przetrenowania (overfittingu), ponieważ wymaga to wyuczenia większej liczby parametrów, co z kolei może przeciwdziałać nadmiernemu dopasowaniu do danych treningowych.
4. **Redukcja wymiarowości**: W przypadku zastosowania operacji poolingowych lub warstw konwolucyjnych o większych rozmiarach zwiększenie liczby kanałów pozwala na zachowanie bogactwa informacji przy jednoczesnej redukcji wymiarowości danych.

Wzrost wymiaru kanału w warstwie wejściowej sieci konwolucyjnej ma kilka zastosowań:

1. **Zróżnicowanie informacji**: Obrazy kolorowe mają trzy kanały (RGB), co oznacza, że są traktowane jako zestawy trzech warstw. Zwiększając wymiar kanału na wejściu można wprowadzić dodatkowe informacje, na przykład warstwę alfa w przypadku obrazów RGBA lub dodatkowe informacje o kolorach (na przykład przestrzeń barw CMYK). To pozwala sieci na lepsze wykorzystanie dostępnych danych.
2. **Wartość semantyczna**: W niektórych przypadkach, zamiast dodawania kanałów reprezentujących różne przestrzenie barw, możemy dodać kanały, które mają wartość semantyczną dla konkretnego problemu. Na przykład, jeśli sieć konwolucyjna analizuje obrazy rentgenowskie, jednym z kanałów w warstwie wejściowej może być maska obszaru zainteresowania, który zawiera obszary ciała, na których ma się skupić analiza.
3. **Uwzględnienie dodatkowych cech**: Czasami, oprócz informacji o kolorach, istnieją inne cechy, które mogą być przydatne dla danego problemu. Może to być głębokość obiektu w przypadku obrazów 3D, prędkość w przypadku obrazów sekwencyjnych itp. Zwiększenie wymiaru kanału pozwala na uwzględnienie tych dodatkowych cech w analizie.

Należy pamiętać, że zwiększenie wymiaru kanału wymaga zwykle większej mocy obliczeniowej i może prowadzić do zwiększenia ilości parametrów modelu, co z kolei może wpłynąć na większe zapotrzebowanie na moc obliczeniową. Dlatego zawsze należy dostosować architekturę sieci do konkretnego problemu i dostępnych zasobów obliczeniowych.